Int J Adv Manuf Technol (2015) 78:1595–1608
DOI 10.1007/s00170-014-6704-8

ORIGINAL ARTICLE

Ontology-based disassembly information system for enhancing disassembly planning and design

Bicheng Zhu · Utpal Roy

Received: 23 September 2013 / Accepted: 11 December 2014 / Published online: 11 January 2015

Abstract Disassembly, as one of the core steps in the end of life (EOL) activities, has been a popular topic both in industrial and academic areas. Unlike assembly, any practical disassembly process planning, which is very important for material recycling or component reuse, is still a topic of active research. One major obstacle in developing an optimal disassembly process planning system is the uncertain characteristics of the EOL products (uncertain information), such as the quality and quantity of the components and parts, which normally make a deterministic disassembly process planning impossible. How to represent and further use such uncertain information in reasoning about a realistic optimal disassembly process sequence are the most important research issues. This paper addresses these issues by developing a disassembly information framework which consists of both an ontology-based information model (domain semantics and rules) and a computational model (process planning generation methodology). The explicit domain concepts, relationships, or rules developed in the ontology-based information enhance the process planning generation methodology (computational model) and thus make the planning process efficient and more realistic. Examples are used in the end to illustrate the use of the proposed framework and methodology.

Keywords Uncertainty representation · EOL · Information model · Sustainable manufacturing · Sequence optimization · Ontology

B. Zhu (✉) · U. Roy
Department for Mechanical and Aerospace Engineering, L.C. Smith College of Engineering and Computer Science, Syracuse University, Syracuse, NY 13244, USA
e-mail: bizhu@syr.edu

1 Introduction

A disassembly process planning system plans for decomposing one product into its parts or components. Most disassembly operators tend to carry out the process as a reverse of the assembly process; however, such approach is normally inefficient or sometimes impossible. One major problem is that the status of the end of life (EOL) products is different from their originally designed products (before their usage), and their EOL status is mostly determined due to the different conditions they went through during their life cycles. For example, the quality of the part or component could be totally uncertain; it might be worse due to years of usage, or it might be in better condition compared to the whole product if any of its old parts were replaced by a new part(s) during its maintenance. This kind of indeterminate characteristics (or say indeterminate information) associated with the EOL product has to be represented and processed systematically in order to make any realistic and credible disassembly process plan. An information system is usually the outcome of such requirements.

One important difference between information systems and information models should be emphasized here. An information model specifies the domain concepts and relationships, and it is declarative (thus, it cannot be run like a program) in nature. It is a data model, and the implementation of such a model is nothing but a text-based static file. The aim of an information model is to formalize and standardize the information about a domain. The National Institute of Standards and Technology (NIST) published several information models [1–4] which aim at developing a formal and well-structured information model for a product with different perspectives. However, how to process the information is rarely emphasized. An information system, on the other hand, takes and utilizes the information data model and goes through some processing algorithms; thus, it comprises a dynamic schema

(we referred to it as a computational model here). The product data management (PDM) and the product life cycle management (PLM) systems are the most popular product-related information systems that are currently available in the market. However, neither of them supports uncertain information representation or processing.

This paper proposes an ontology-based information system for automated disassembly planning. Its goal is to develop a formal and structured ontology-based information model (data model) which handles disassembly concepts, relationships, rules, as well as uncertain characteristics, and also to develop a computational model for the generation of the appropriate disassembly plans.

2 Literature review

A disassembly process planning problem involves three major issues related to (a) product representation, (b) disassembly sequence optimization, and (c) uncertain elements integration. A lot of research has been done on all these issues. Sections 2.1 and 2.2 briefly review issues related to (a) and (b). In order to address the issue related to (c), this paper uses the fuzzy logic theory proposed by Zadeh [5] to handle uncertain information in a disassembly process. Thus, Section 2.3 briefly introduces the fuzzy logic theory, together with some other approaches to handle disassembly uncertainty.

2.1 Product representation

In a product disassembly system, modeling the product topology and geometry information is important, especially for generating the disassembly sequences. A number of graph-based modeling strategies have been used before, such as AND/OR graph [6, 7], directed graph [6, 7], disassembly petri net (DPN) [7–9], and so on. Among all of them, the AND/OR graph proposed by Homem De Mello [6] is widely used in this work, and thus, a detailed explanation is necessary here for the understanding of this reported work.

An AND/OR graph is a directed graph $G=(N, D)$, where N stands for nodes that denote a product, part, or subassembly. D stands for hyper arcs which represent the set of feasible disassembly operations. (Each node can have k ($k\geq 1$) disassembly methods, forming an OR relation; if an operation disassembles node i into m ($m>2$) nodes, m arcs link node i to these m nodes and form an AND relation.) Figure 1 is a simple example of the AND/OR graph of a product. Arc 1 in the figure represents disassembly operation 1, and assembly ABCDE can be disassembled into subassembly ABCD and part E (which is not shown in Fig. 1) through disassembly operation 1. Similarly, operation 3 disassembles subassembly ABCD into subassembly AB and subassembly CD. Each path in the AND/OR graph forms a feasible disassembly sequence.

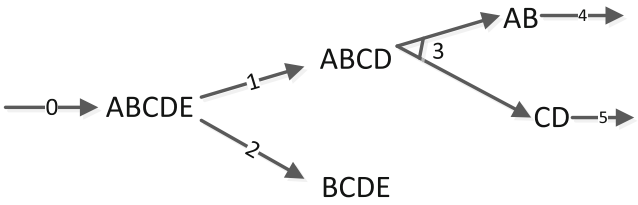

Fig. 1 An example of an AND/OR graph

In Fig. 1, for example, path 0-1-3-4-5 is one of the feasible disassembly sequences. (Operation 0 represents the initialization of the disassembly process.)

One shortcoming of the AND/OR graph representation is that the disassembly sequence is implicit in the graph; for example, unlike path 0-1-3-4-5, path 0-1-3-5-4 is also a viable disassembly sequence. However, we cannot identify that easily in an AND/OR graph, especially when the graph becomes complicated.

One variation to solve this problem is using a so-called task precedence graph in which nodes represent disassembly operations and two disassembly operations are represented by two nodes connected by a directed arc signifying that one operation proceeded by the other. If the AND/OR graph in Fig. 1 is translated into a task precedence graph, it will look like Fig. 2 below. As we can see from Fig. 2, each possible disassembly path (each viable path is a disassembly sequence) can be identified in the graph.

2.2 Disassembly sequence optimization

Based on the AND/OR graph and its variation, the task precedence graph, a linear programming (LP)-based disassembly sequence optimization model, and some extensions have been proposed [10–12]. The LP model gives the optimal solution based on maximizing the total value (cost) of the retrieved part/component and minimizing the total disassembly cost associated with them. Taking Fig. 1 as an example, if we assign each disassembly operation (1, 2, 3, 4, and 5) to a binary decision

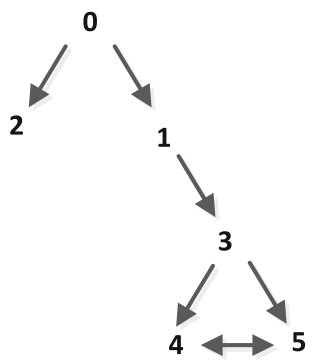

Fig. 2 The task precedence graph of Fig. 1

variable ($x1$, $x2$, $x3$, $x4$, $x5$), the value we can retrieve from a sequence of disassembly operations is

$$\begin{aligned}\text{Value} = {} & V_{\text{ABCDE}} \times (x0{-}x1{-}x2) + V_{\text{ABCD}} \times (x1{-}x3) \\ & + V_{\text{BCDE}} \times (x2) + V_{\text{AB}} \times (x3{-}x4) + V_{\text{CD}} \\ & \times (x3{-}x5) + V_{\text{A}} \times (x2 + x4) + V_{\text{B}} \times (x4) + V_{\text{C}} \\ & \times (x5) + V_{\text{D}} \times (x5)\end{aligned}$$

where V_i is the value of component/part i and xi is the binary decision variable for disassembly operation i.

If we do only operation 0, 1, 3, and 4 (i.e., $x0{=}x1{=}x3{=}x4{=}1$, others equal to 0), the above equation shows us that the total value we can retrieve from such disassembly operations is

$$\begin{aligned}& V_{\text{ABCDE}} \times (1{-}1{-}0) + V_{\text{ABCD}} \times (1{-}1) + V_{\text{BCDE}} \times (0) \\ & + V_{\text{AB}} \times (1{-}1) + V_{\text{CD}} \times (1{-}0) + V_{\text{A}} \times (0 + 1) + V_{\text{B}} \\ & \times (1) + V_{\text{C}} \times (0) + V_{\text{D}} \times (0) \\ & = V_{\text{CD}} + V_{\text{A}} + V_{\text{B}}\end{aligned}$$

We can represent the above as a generalized formulation as follows:

$$V = \sum_i \sum_j V_i \times T_{i,j} \times x_j$$

where T is the coefficient matrix and the value of the element in the matrix ($T_{i,j}$) is equal to −1, 0, or 1. The subscript j corresponds to operation and subscript i corresponds to part or subassembly. If operation j disassembles subassembly i, $T_{i,j}{=}{-}1$. If operation j assembles part i into a subassembly, $T_{i,j}{=}1$. For other conditions, $T_{i,j}{=}0$. For the example in Fig. 1, the T matrix is as follows:

	0	1	2	3	4	5
ABCDE	1	−1	−1	0	0	0
ABCD	0	1	0	−1	0	0
BCDE	0	0	1	0	0	0
AB	0	0	0	1	−1	0
CD	0	0	0	1	0	−1
A	0	0	1	0	1	0
B	0	0	0	0	1	0
C	0	0	0	0	0	1
D	0	0	0	0	0	1
E	0	0	0	0	0	0

Following the same analysis for the disassembly operation cost, the complete disassembly LP model formulation is as follows:

Objective function:

$$V{-}C = \sum_i \sum_j V_i \times T_{i,j} \times x_j {-} \sum_{j,k} C_{j,k} \times y_{j,k}$$

Constraints:

1. $\sum x_{i,\text{in}} \geq \sum x_{i,\text{out}}$
2. $\sum x_{i,\text{in}} \leq 1$
3. $x_0 = 1$
4. $x_j = \sum_k y_{k,j}$
5. $\sum_k y_{k,j} = \sum_k y_{j,k}$

Decision variables are x_j and $y_{j,k}$, whereas V_i and $C_{j,k}$ are constant coefficients representing the value of each part/component and the cost of the disassembly operation. Decision variable x_j corresponds to each arc in the AND/OR graph, and $y_{j,k}$ corresponds to each arc in the task precedence graph. All of the decision variables are binary variables.

Applying the LP model, one optimal disassembly sequence will be generated. (For example, the optimal sequence for the example in Fig. 1 is 0-1-3-4-5.)

One of the biggest assumptions in the traditional LP model is that the value of the part/component is known and deterministic which is not true for the end of life product. How to evaluate such uncertain information is one of the major thrusts of this work.

2.3 Disassembly uncertainty and fuzzy logic

Some research has been done [13–15] to deal with the uncertain information associated with a disassembly process. For example, Zhou and Zussman [13] proposed a disassembly petri net (DPN) for modeling and adaptively planning the disassembly processes. Guangdong et al. [16] suggested a probability model to evaluate the disassembly cost which was subjected to random removal time and different removal labor costs. This paper focuses on tackling the uncertainty problem from an information modeling perspective. Current approaches or methodologies are quite difficult to integrate into an information model since the information is too specific to a certain method and the semantics of the information is rarely emphasized or noticed.

In this work, we propose a computational model for disassembly planning analysis which utilizes the semantics of the proposed disassembly information model. The details of the methodology are given in Section 3. The proposed computational model embeds the traditional LP model and fuzzy logic theory. Thus, a brief review on fuzzy logic theory [5] is necessary here.

Fuzzy logic is a form of many-valued logic and deals with reasoning that is approximate rather than fixed and exact.

Compared to traditional binary sets (where variables may take on true or false values), fuzzy logic variables may have a truth value that ranges between 0 and 1. Fuzzy logic can be extended to handle the concept of partial truths, where the truth degree (TD) may range between completely true and completely false. Fuzzy logic has several important elements which include

(1) Linguistic variable

Fuzzy logic usually associates linguistic variables rather than numerical variables. A linguistic variable differs from a numerical variable in that its values are not numbers but words or sentences in a natural language. For example, "part/component value" can be a linguistic variable; its value is V={low, medium, and high}.

(2) Membership function

In order to map the values of a linguistic variable to truth degrees whose value ranges from 0 to 1, a membership function is needed. As an example, the membership function of the linguistic variable "part/component value" for part A can be defined as shown in Fig. 3 below.

Figure 3 shows the membership function for each of the values of the linguistic variables (low, medium, and high). For example, if the value of part A is $60, TD values for "low" and "high" would be equal to zero, while the TD value for the "medium" would be equal to one. It means that the value of part A can be considered to be medium to a degree of 1, whereas it can be considered to be low or high to a degree of 0.

(3) Fuzzy control rules

Fuzzy rules are a set of user-defined rules that are used to infer the value of output variables and take the following forms:

IF <*Antecedent*> then <*Consequent*>. Two fuzzy rule examples are as follows:

R1: If <*Experience* is little> and <*PartAge* is old>, then <*PartValue* is little>.

R2: If <*Experience* is little> and <*PartAge* is medium>, then <*PartValue* is low>.

Experience, *PartAge*, and *PartValue* are linguistic variables. Fuzzy rules can be defined and modified with human interactions, so they are flexible.

The key idea of fuzzy logic inferring is that the crisp input information is translated to some fuzzy numbers first and uses an inference engine to generate the fuzzy value of the output variables. The inference engine comprises all the fuzzy rules that the user defines. After that, the fuzzy output will again be translated into crisp values using the center of gravity theory, and thus, the final output consists of the crisp value only.

Such inferring process can be done in the MATLAB Fuzzy Toolbox. Figure 4 shows an example of inferring the value of part A.

As shown in Fig. 4, the crisp input is [*OperatorExperience*=15, *partAge*=5]. Nine fuzzy control rules are defined (not shown in Fig. 4), and each generates the fuzzy value of the output variable (*PartValue-A*). The nine generated fuzzy values (for the output) are then aggregated and translated into a crisp value (by using center of gravity theory); the final inferred result is 60.4.

3 Ontology-based disassembly information system: the framework

In this section, we describe the proposed ontology-based disassembly information system framework and how it can help disassembly process planning. The proposed ontology-based disassembly information system consists of two interrelated models: information model and computational model. More specifically, the information model development goes through three major steps:

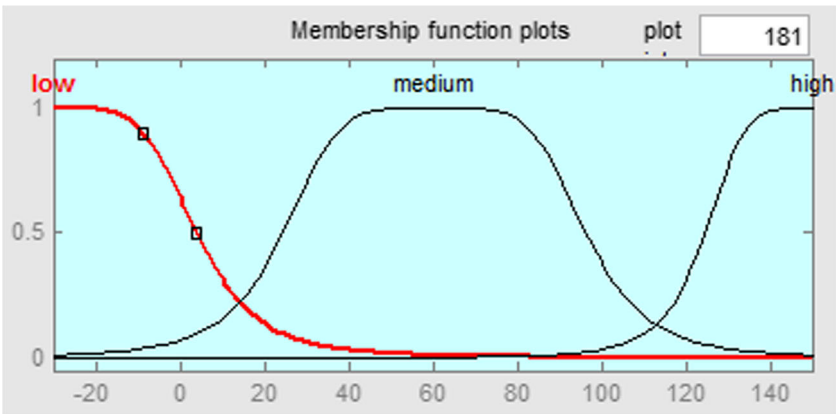

Fig. 3 A membership function example

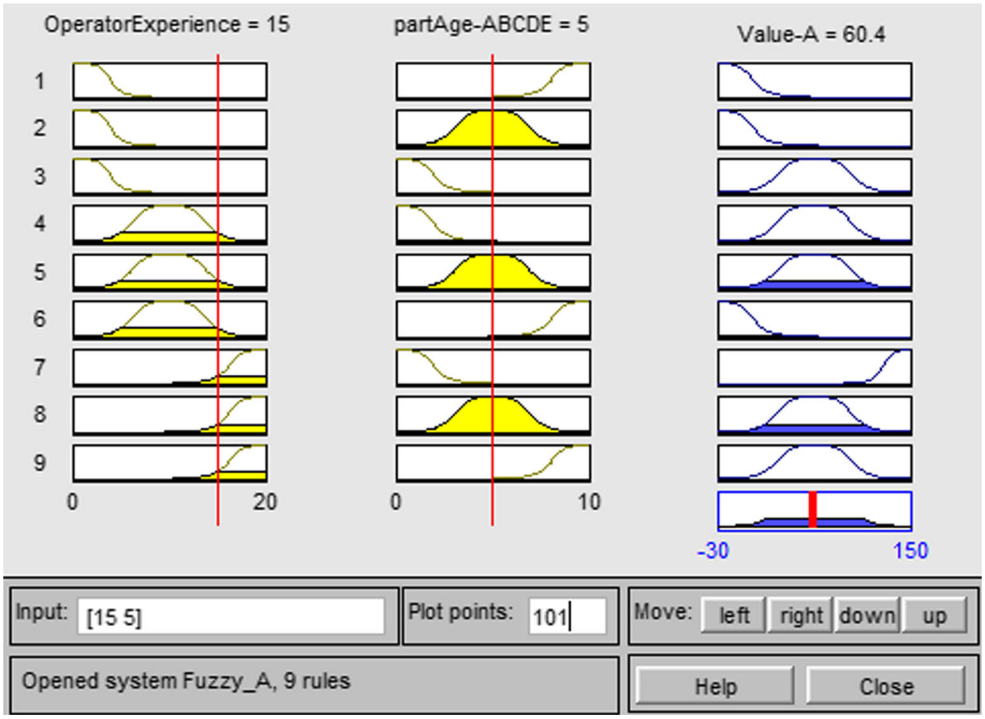

Fig. 4 A fuzzy inferring example in MATLAB

(1) Knowledge source acquisition. As it is evident from Fig. 5, a variety of knowledge sources is collected, including design specification, end of life statistical information, design rules, etc. This knowledge is originally in different format and is scattered or disorganized.

(2) Knowledge modeling. The disorganized information collected in the previous steps is generalized into formalized domain concepts and relationships. Such model is neutral and can be further realized into a different format.

(3) Model realization. The formalized model is realized in Protégé, an ontology development tool. Since we need to

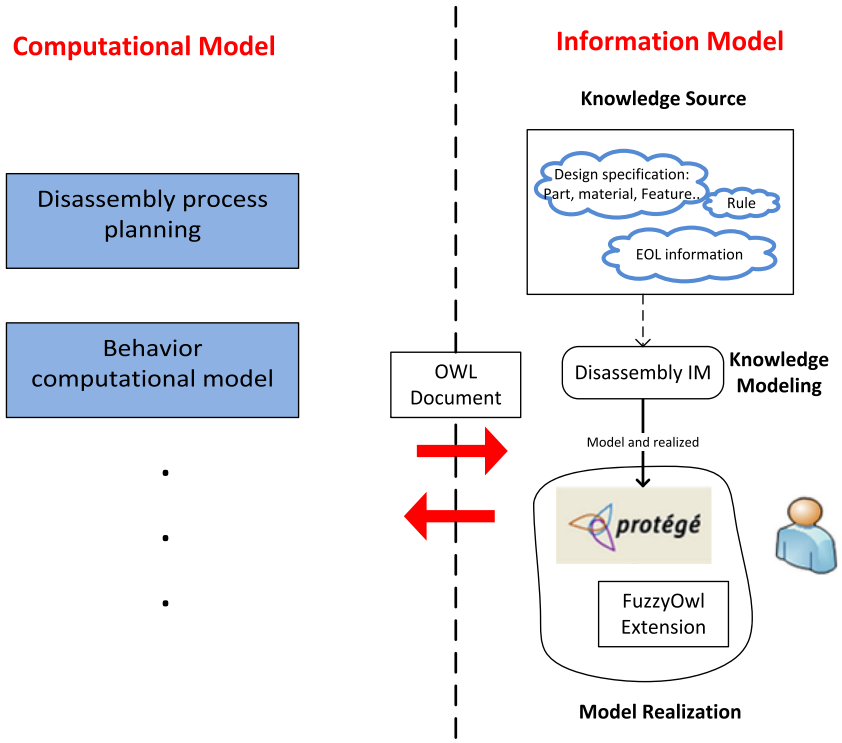

Fig. 5 Ontology-based disassembly information system framework

handle uncertain information regarding an EOL product, such information should be included. Standard ontology does not support uncertain information representation and reasoning. We are going to use an extension method proposed by Bobillo and Straccia [17] to encode the uncertain information as annotation properties.

The computational model serves the purpose of processing the information model output (OWL document) and providing specific results. In other words, the computational model is objective specific and algorithm based. Some research [18] has been done on one of the vital aspect of the computational model: behavior computational model, which will update the information model instance base according to the behavior of a certain domain.

In this research, our final objective is to help solve the disassembly process planning; thus, a specific computational model is developed for that purpose. The detailed algorithm is introduced in the following sections. Here, a higher-level architecture of the computational model is shown in Fig. 6. The computational model utilizes several key components as follows for the disassembly plan generation:

(1) Apache Jena. Parse the OWL file and get the information needed.
(2) Ontology reasoner. Semantic query. A key step for the disassembly process planning algorithm. (Described in detail in the following section)
(3) Fuzzy logic. Theory for handling the uncertain or fuzzy information. This paper uses MATLAB Fuzzy Toolbox for fuzzy logic implementation.

The following sections are organized as follows: Section 3.1 presents the disassembly information model (data model, DIM); Section 3.2 explains the realization of the DIM in ontology by utilizing Protégé; and Section 3.3 thoroughly describes the computational model for disassembly process planning.

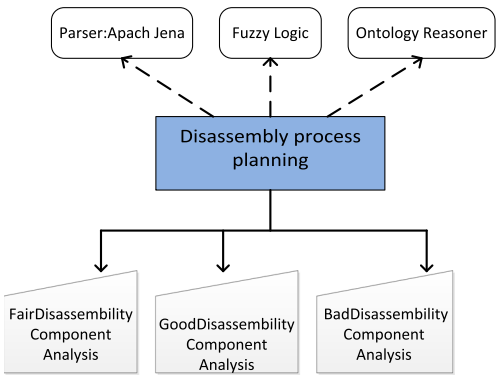

Fig. 6 Disassembly process planning computational model architecture

3.1 Proposed disassembly information model

The structure of the proposed disassembly information model (DIM) is shown in Fig. 7 as a UML (Unified Modeling Language) class diagram. DIM is developed to represent the following major information or knowledge in a disassembly process:

(1) Domain concepts and relationships. Information is generalized into concepts or say classes (denoted as a rectangular box with a concept name in Fig. 7), and the relationship between the concepts is identified. (*Composition* relationship is denoted as a filled diamond arrow and *inheritance* relationship is denoted as a hollowed triangular arrow.)

As an example, the major concepts in a disassembly process are *Part*, *Component*, *Material*, *Process*, etc., and the relationship between a *Part* and a *Component* is *Composition*, which means a *Component* instance comprises several *Part* instances.

Semantics of the classes (for example, what is *CompositePart*?) is implicit in the class diagram and will be described in detail in the ontology realization section.

(2) EOL knowledge. In order to represent and further solve the disassembly uncertainty analysis, EOL statistical knowledge is included in the DIM. One of the key uncertain information in a disassembly process is the *Remaining Value* of an EOL part or component, which is vital in a disassembly planning process. The strategy to get this information in this paper (explained in detail in Section 3.3) is to infer the *Remaining Value* from the age of the part/component and the experience of the operator. In order to do such inferring, some EOL knowledge is needed (*EOLAge*, *EOLValue*, *OperatorExperience*, etc. as for an example).

As a simple example here, an instance of *EOLAge* contains the following information:

Name	"EOLAge-ABCDE"
Range	[0 10]
Number of MFs	3
MF1	"Young":"gbellmf," [2 2.5 0]
MF2	"Middle":"gbellmf," [2 2.5 5]
MF3	"Old":"gbellmf," [2 2.5 10]

As we can see, the instance information of the class *EOLAge* gives us the name (EOLAge-ABCDE), the age range ([0 10]), and the membership function (MF1, MF2, MF3) of the component ABCDE. For example, we can consider the age of component ABCDE as *Young* following a generalized bell-shaped membership function (*gbellmf*) with parameters [2 2.5 0].

EOLValue and *OperatorExperience* class follow the same information structure as *EOLAge*. *Fuzzy_Control_Rule, on the*

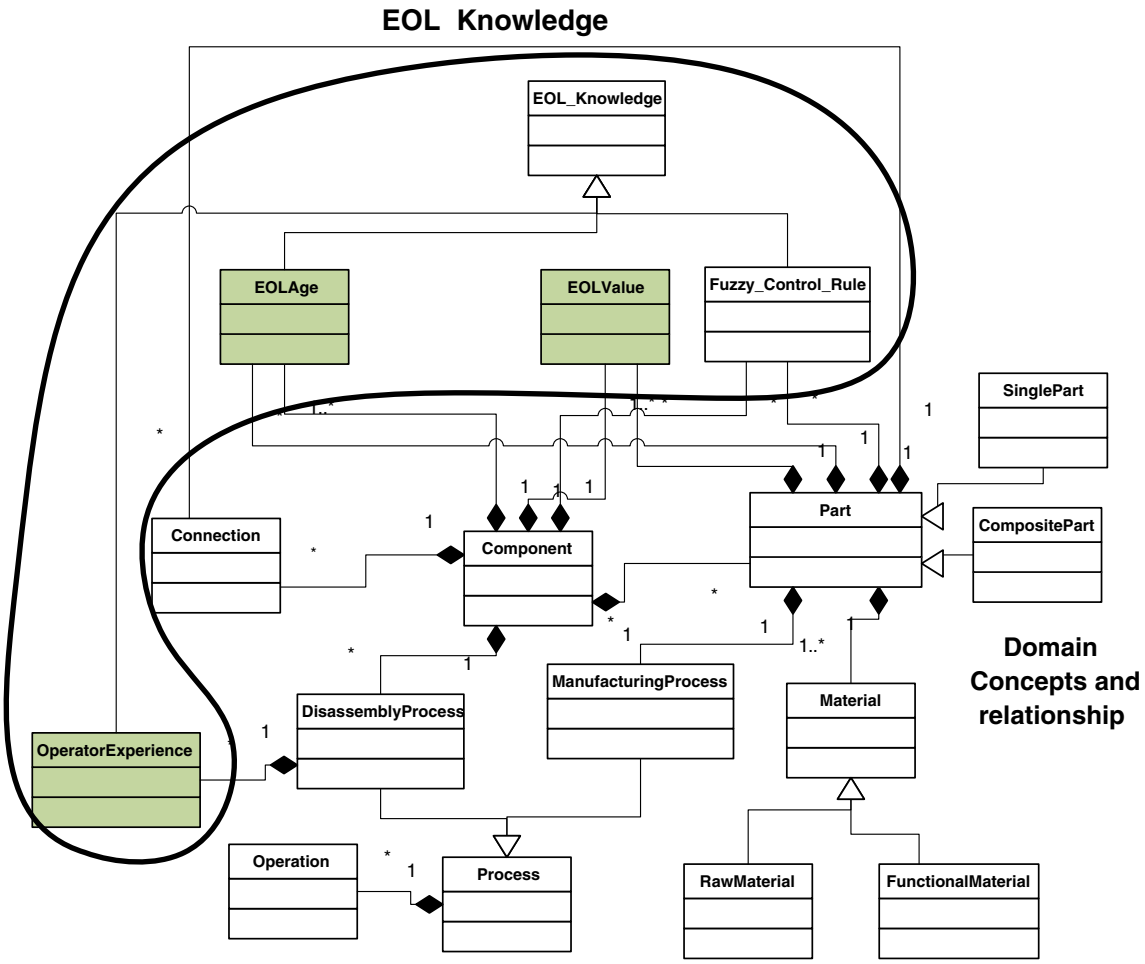

Fig. 7 Disassembly information model developed in the UML class diagram

other hand, is a special EOL knowledge; it tells us how we want to control the inferring process through which the value of the part/component can be obtained. It is based on the statistical result from years of practice of a certain product disassembly. One instance information can be as follows:

Name	"Fuzzy_Control_Rule-ABCDE"
Control	Value of ABCDE
Rule	If (OperatorExperience is Little) and (EOLAge-ABCDE is Old) then (EOLValue-ABCDE is Low)

The above *Fuzzy_Control_Rule* defines what the value of ABCDE is expected (Low) if *OperatorExperience is Little and EOLAge is Old*. All the EOL knowledge is crucial for the uncertain disassembly process generation.

3.2 Implementation of the DIM in ontology

The DIM model is further realized in ontology by using Protégé as an ontology development tool. The result of the ontology-based disassembly information model is an OWL file which can be processed by the computational model as described in Section 3.3. Ontology usually is a 4-tuple, O= <C, P, R, A> [19], where

(1) C is a set of concepts defined in a domain.
(2) P is a set of concept properties. A concept property can have a type restriction, cardinality restriction, and range restriction. Basic type restriction can be {Boolean, integer}; cardinality restriction defines the upper and lower limits on the number of values for the property; the range restriction specifies a range of values that can be assigned to the property.
(3) R is a set of binary semantic relations defined between concepts in C. Basic relations can be {kind of, part of, property of}.
(4) A is a set of axioms. An axiom is a real fact or reasoning rule.

C, P, and R have a corresponding mapping to the proposed DIM; however, axioms or rules (A) are inexplicit in the original DIM, which means DIM represented in the UML class diagram tells us about the necessary condition of being a certain instance of a class

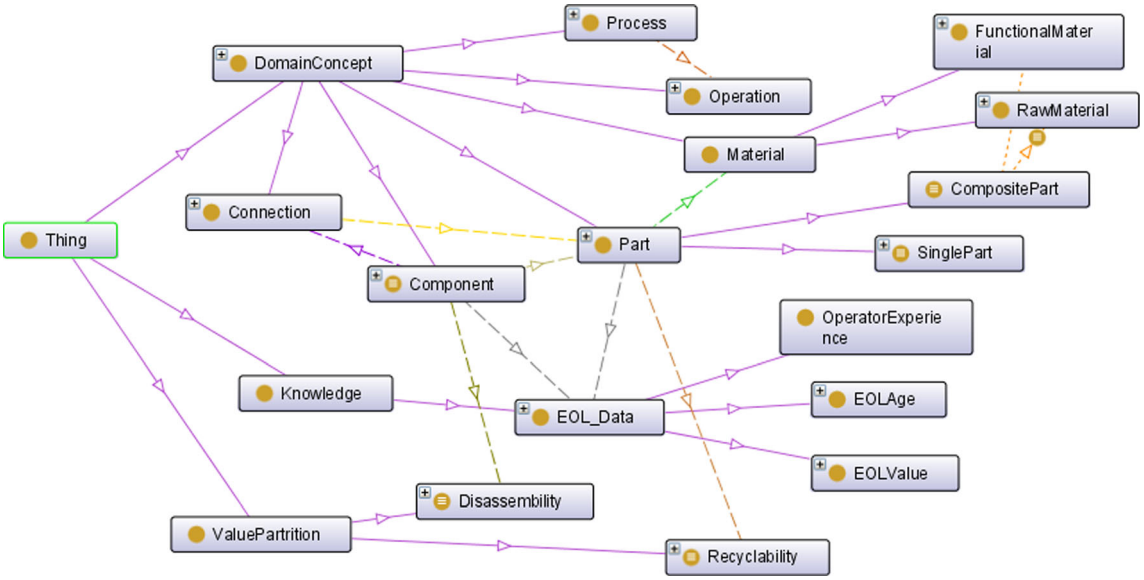

Fig. 8 DIM ontology realization in Protégé

(what are the attributes, what is the relationship with other classes, etc.); it does not inform what makes an object sufficient to belong to the class. In other words, ontological realization of the proposed DIM extends and

Table 1 Rules to fully define the semantics of DIM

Rule	Semantics	Example/explanation
1. Component=hasPart min 2 Part	A Component is anything which has 2 parts	Subassembly in a product
2. SinglePart=Part and((hasMaterial exactly 0 FunctionalMaterial) and (hasMaterial exactly 1 RawMaterial))	Anything which is a part and has no FunctionalMaterial and has only 1 RawMaterial is SinglePart	Any part which contains only one material like plastic and goes through no extra processing such as surface finishing
3. CompositePart=Part and ((hasMaterial min 1 FunctionalMaterial) and (hasMaterial min 1 RawMaterial))	Anything which is a part and has at least 1 FunctionalMaterial and RawMaterial is CompositePart	Part with chromium plating, part with two materials
4. Disassembility={BadDisassembility, FairDisassembility, GoodDisassembility}	Disassembility can be one of the three possibilities: bad, fair or good	N/A
5. Recyclability={BadRecyclability, FairRecyclability, GoodRecyclability}	Recyclability can be one of the three possibilities: bad, fair or good	N/A
6. SinglePart(?x)→hasRecyclability(?x, GoodRecyclability)	If x is a SinglePart, it has GoodRecyclability	Recycle process only needs to deal with one raw material
7. Part(?x), FunctionalMaterial(?m2), RawMaterial(?m1), (hasMaterial exactly 1 RawMaterial)(?x), hasMaterial(?x, ?m1), hasMaterial(?x, ?m2)→hasRecyclability(?x, FairRecyclability)	If x is a Part, and it has only 1 Raw material, but has several functional material, then it has FairRecyclability	Recycle process needs to deal with the functional material like painting on the part before extracting the main material
8. FunctionalMaterial(?y), Part(?x), RawMaterial(?m1), RawMaterial(?m2), hasMaterial(?x, ?m1), hasMaterial(?x, ?m2), hasMaterial(?x, ?y), DifferentFrom (?m1, ?m2)→hasRecyclability(?x, BadRecyclability)	If x is a Part, and it has more than 1 Raw material, and it has at least 1 functional material, then it has BadRecyclability	Recycle process needs to deal with the functional material like painting on the part and then needs to separate the different main materials
9. Component(?x), (hasConnection only DetachableConnection)(?x)→hasDisassembility(?x, GoodDisassembility)	If x is a Component, and it has only DetachableConnection, then it has GoodDisassembility	A subassembly which can be totally disassembled without breaking the parts
10. Component(?x), (hasConnection some DetachableConnection)(?x), (hasConnection some IrreversibleConnection)(?x)→hasDisassembility(?x, FairDisassembility)	If x is a Component, and it has DetachableConnection and IrreversibleConnection, then it has FairDisassembility	A subassembly which cannot be totally disassembled without breaking a part
11. Component(?x), (hasConnection only IrreversibleConnection)(?x)→hasDisassembility(?x, BadDisassembility)	If x is a Component, and it has only IrreversibleConnection, then it has BadDisassembility	A subassembly which cannot be totally disassembled without breaking all the parts

enriches the semantics of the disassembly system. With such enrichment, new facts or information which did not reside originally in the ontological information base can be inferred.

The ontology-based implementation of the DIM is shown in Fig. 8.

The dotted line in Fig. 8 illustrates a *has-a* relationship and the normal line describes an *is-a* relationship. The DIM ontology implementation represents a graph structure; however, axioms are needed to fully define the semantics of the model. Table 1 lists the axioms defined in the DIM which are implicit in the DIM's UML diagram.

By adding the rules as shown in Table 1, the ontology file can be reasoned to get new facts which is one of the key advantages for an ontology-based information model. This kind of reasoning is vital to the proposed computational model described in the next section. Figure 9 below shows a reasoning example.

As we can see from Fig. 9, before the reasoning process, the fact about instance A is that A is a Part, and A contains Material ABS, PMMA, and chromium plating. ABS and PMMA are *RawMaterial*s and chromium plating is *FunctionalMaterial* (defined in each instance definition dialog box, not shown in Fig. 9). With these facts, the reasoner (Pallet) will utilize defined rule no. 1 and rule no. 8 and give an inferred result: A is a *Composite* and A has *BadRecyclability*, as shown in Fig. 9b.

3.3 Proposed computational model for the disassembly process planning

The proposed computational model utilizes the reasoning capabilities of the ontology-based disassembly information model, and it can solve the uncertain disassembly process planning problem by combining traditional deterministic disassembly LP formulation with the fuzzy logic theory. The detailed algorithm for the proposed computational model for disassembly process planning is shown in Fig. 10 below.

The algorithm starts with evaluating the current EOL product and locating the corresponding *Component* in the DIM (OWL file). Then, the inference engine gets the *Disassembility* of this *Component* by the semantics reasoning process. Based on the result from the reasoning process (*GoodDisassembility*, *FairDisassembility*, or

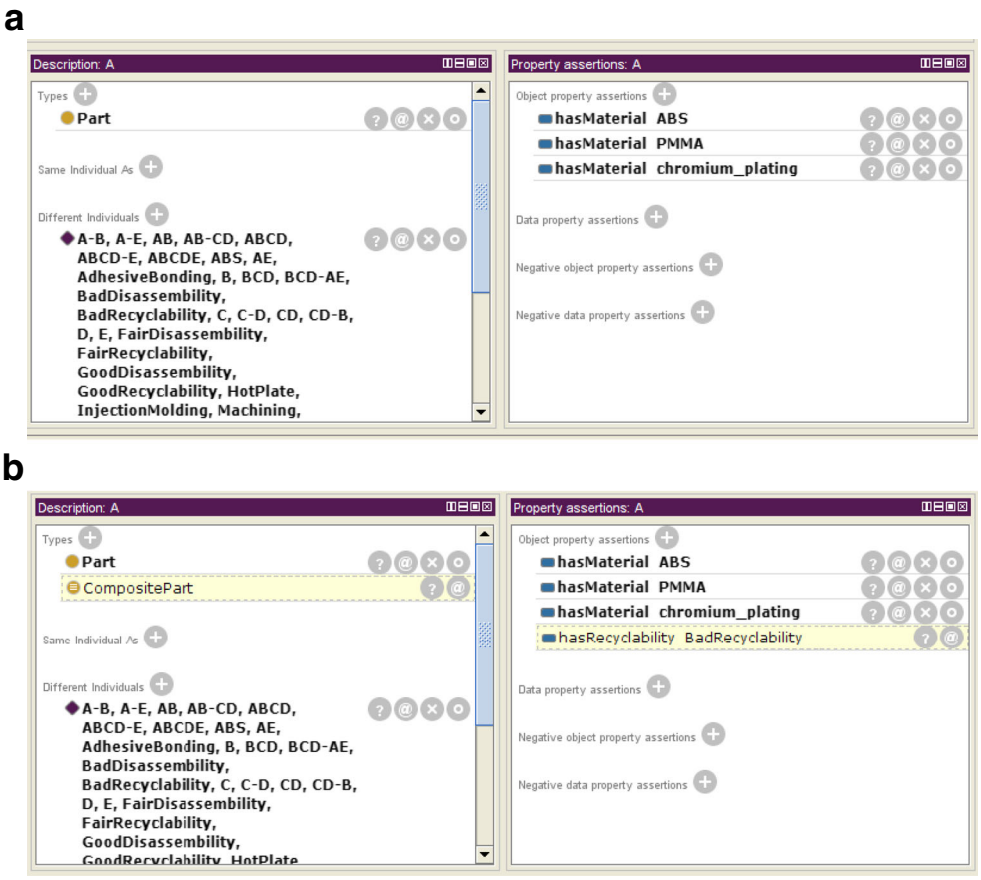

Fig. 9 A reasoning example. **a** Fact about instance A before reasoning. **b** Fact about instance A after reasoning

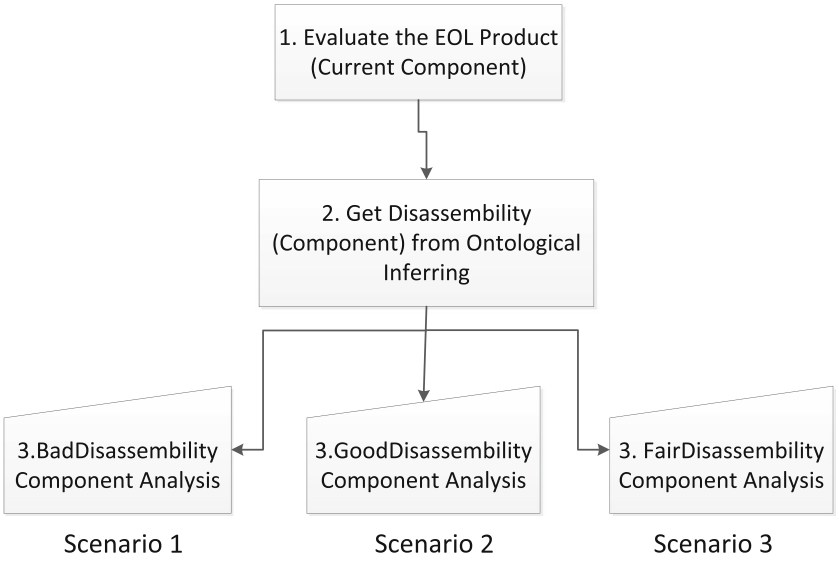

Fig. 10 The computational model for disassembly process planning

BadDisassembility), different detailed algorithms are proposed in Figs. 11, 12, and 13.

Scenario 1: Component has *BadDisassembility*, which means all the *Connections* that the *Component* has are *IrreversibleConnection*s. In other words, the *Component* cannot be totally disassembled without breaking all the parts. Thus, the disassembly planning strategy for such *Component* is to recycle the parts instead of reusing the part/component.

In order to identify the best disassembly sequence for recycling parts, we need to identify the candidate *Part* which is going to be recycled. This is done by the semantic reasoning on parts' *Recyclability* (as we did with *Disassembility* before). After the reasoning process, if the *Component* has no *Part* which has *GoodRecyclability*, the *Component* will be discarded without carrying out any further disassembly operation. If the *Component* has a *Part* which has *GoodRecyclability*, the Linear Programming (LP) model is modified to make sure such Part is completely retrieved after the disassembly process. More specifically, we need to set the corresponding operation binary variable to 1 and all the *Component* value is equal to 0 and *Part* value to recycle value. After running the modified LP model, a disassembly process path will be generated, and the Part retrieved from the process will be considered to be recycled.

Scenario 2: *Component* has *GoodDisassembility*, which means all the *Connections* that the *Component* has are *DetachableConnection*. In other words, the *Component* can be totally disassembled without breaking any part. Thus, the disassembly planning strategy for such *Component* is to consider *Component/Part* reuse first, then *Part* recycle.

The above algorithm comprised two main components (a fuzzy ontology engine and a LP model). The LP model generates the optimal disassembly sequence from a product structure graph like an AND/OR graph. The fuzzy ontology engine handles the uncertain information in the disassembly

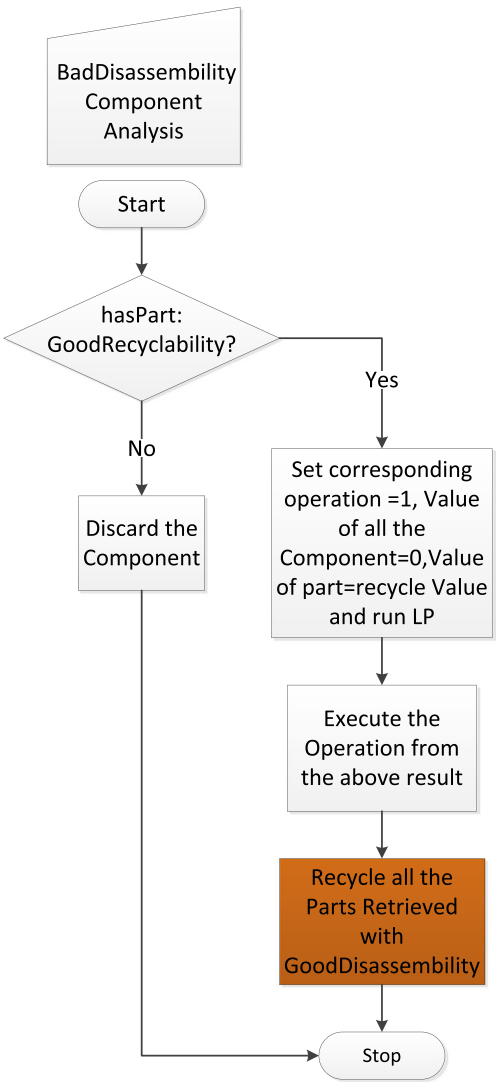

Fig. 11 *BadDisassembility* component analysis

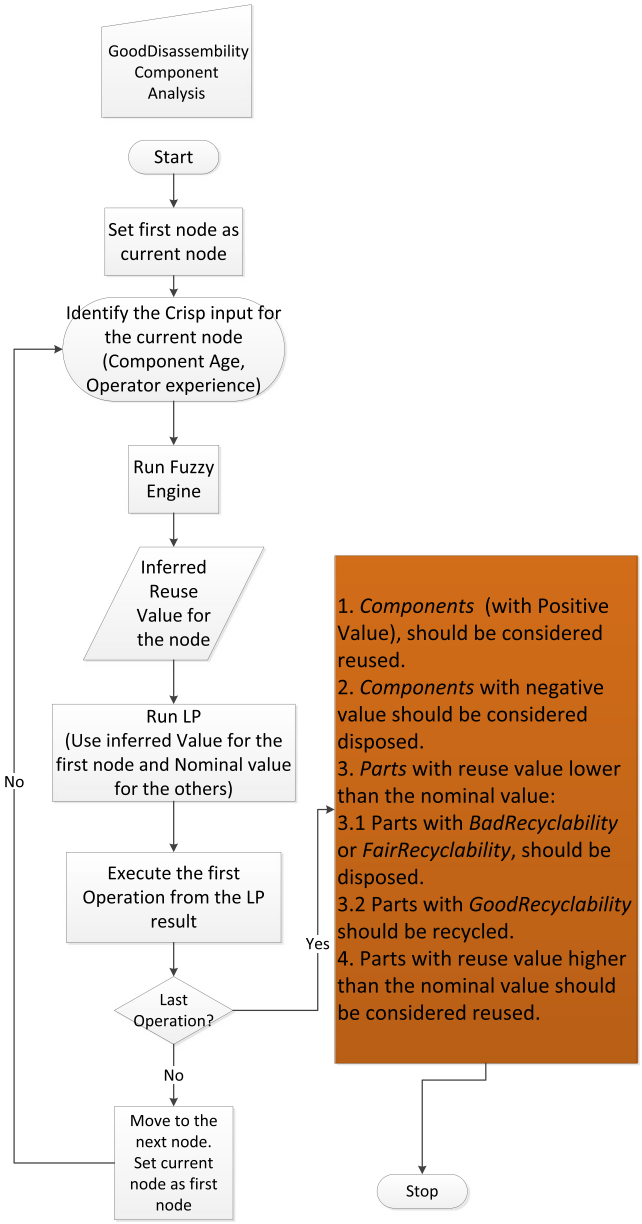

Fig. 12 *GoodDisassembility* component analysis

process, and it generates the critical input like part values, which the LP model needs. The algorithm runs recursively and finds the optimal disassembly sequence under uncertainty dynamically. The detailed explanation on the proposed algorithm can be found in [20].

The result from the algorithm is further analyzed with the original data semantics embedded in the disassembly information model, and the detailed disassembly strategy (recycle or reuse) is carried out accordingly, as shown in the shaded box in Fig. 12. For retrieved *Component*, if the inferred value for the *Component* is positive, it should be considered to be reused; if negative, it should be considered to be disposed. For the retrieved *Part*, if the inferred value is lower than the nominal value, it should be considered to be recycled or

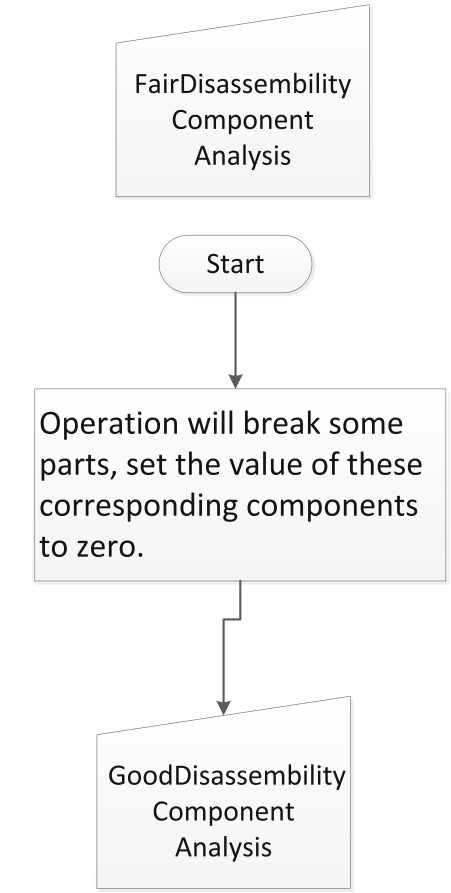

Fig. 13 *FairDisassembility* component analysis

disposed based on its *Recyclability*; otherwise, it should be considered to be reused.

Scenario 3: *Component* has *FairDisassembility*, which means the *Component* has both *DetachableConnection* and *IrreversibleConnection*. The total disassembly of such *Component* will break some parts. Thus, the disassembly planning strategy for this type of *Component* is to recycle the broken *Part* and reuse the unbroken *Part/Component*.

For *FairDisassembility Component* analysis, it is similar to the *GoodDisassembility* analysis, except that the nominal value of the "to be broken" Part/Component (the result of an irreversible disassembly operation) will be set to 0. After that the same analysis as the *GoodDisassembility* will continue.

The proposed computational model for the disassembly process planning has been implemented by utilizing Lingo and MATLAB Fuzzy Toolbox. The implementation of the

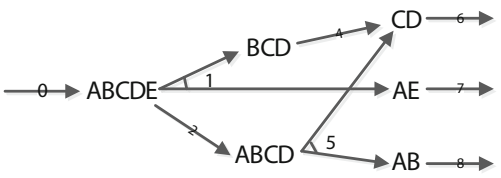

Fig. 14 AND/OR graph of the product ABCDE

Table 2 Nominal value for part/component

Part/component	Reuse value	Recycle value
ABCDE	−35	N/A
ABCD	30	N/A
BCD	−30	N/A
AB	70	N/A
AE	120	N/A
CD	100	N/A
A	50	50
B	60	60
C	20	20
D	−10	10
E	50	50

algorithms has been illustrated with a case study in its following section.

4 A case study

A product with five parts (A, B, C, D, and E) is studied here to demonstrate how the proposed framework can help disassembly planning. The AND/OR graph of the product ABCDE is shown in Fig. 14. The disassembly process planning problem is to define which path we should follow to get the best profit out of a disassembly process and what we should do with the retrieved part/component.

The nominal values of the Part/Component are listed in Table 2.

The information about the product (ABCD) was first stored in Protégé according to the proposed DIM which will be used by the proposed computational model to generate the disassembly planning adaptively. Figure 15 is a screenshot of the instance storage in Protégé.

When applying the proposed computational model for disassembly planning, the first step is to infer the *Disassembility* of the *Component ABCDE*. For that, we consider three different scenarios as follows:

(1) BadDisassembility

For *BadDisassembility* scenario, we first need to infer from the information base and check if the component has any part which is *GoodRecyclability*; the result shows only part B is *GoodRecyclability* as shown in Fig. 16 below.

After that, we will modify the LP model to make sure part B is retrieved from the disassembly process. This is done by adding constraint as follows:

$$X8 = 1$$

The result from the modified LP is 0-2-5-8-6. The objective is equal to 185. The *Part/Component* retrieved from the process is A, B, C, D, and E. Due to the recyclability inference results from the previous step, the post process will recycle part B and discard all the other retrieved parts.

(2) GoodDisassembility

Component (ABCDE) has *GoodDisassembility*, which means all the *Connections* that *Component* (ABCDE) has are *DetachableConnection* and the disassembly planning strategy for such *Component*

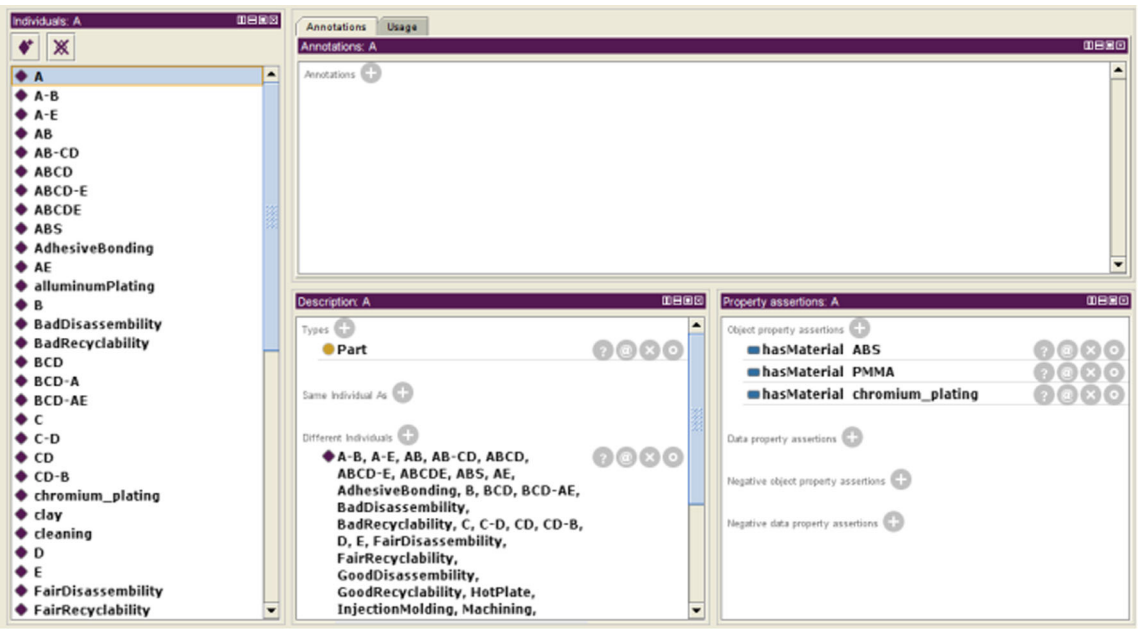

Fig. 15 Instance storage in Protégé

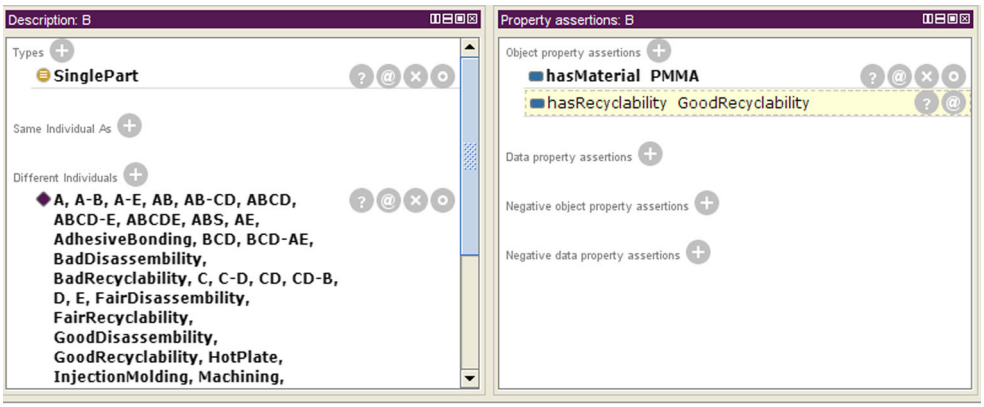

Fig. 16 Inferring *GoodRecyclability* of part B

is to consider *Component/Part* reuse first, then *Part* recycle.

At the beginning, the value for node ABCDE is generated using the fuzzy engine. The values for all the other nodes or arcs are assumed to be nominal values (Table 2) at this time.

The value for node ABCDE is inferred by identifying the crisp inputs (age of the *Component*, 5, and experience of the operator, 10). The inferred value for ABCDE is then −56.3 instead of the nominal value (−35).

Then, we can run the LP model and get one optimal disassembly path as shown in Fig. 13a below (0-2-5). The objective is equal to 195.

Following the optimal path generated above and executing the disassembly operation 2, the disassembly planning process moves to the next iteration. At this stage, because we now know the condition of the subassembly ABCD, we can identify the part value for ABCD (12.7) through the fuzzy engine. This information was considered nominal (30) at the first iteration previously. Again, we need to run the LP model using the updated information for node ABCD. Another optimal path will be generated (happens to be the same as before here) as shown in Fig. 17b below (0-2-5).

Again, after the optimal path generated above and the execution of the disassembly operation no. 5, the disassembly planning process moves to the third iteration. At this stage, because we now know the condition of the subassembly AB and CD, we can identify the values for AB (70.1) and CD (46.8) through the fuzzy engine. The updated information is again sent to the LP model, and another optimal path will be generated as shown in Fig. 17c above (0-2-5-8-6). Since there are no other options after operation no. 8 and no. 6, the disassembly planning is terminated. The retrieved parts are A, B, C, D, and E. For part A, B, C, and E, since the nominal reused values for them are positive, these should be considered for reuse. For part D, since the reuse value is negative (−10), which means it normally has no reuse value and has negative impact on the environment, part D is considered for recycling. However, when checking the information base, we can infer that the recyclability of part D is *BadRecyclability*; thus part D will be disposed.

(3) FairDisassembility

If the *Component* has *FairDisassembility*, it is similar to the *GoodDisassembility* analysis, except that the

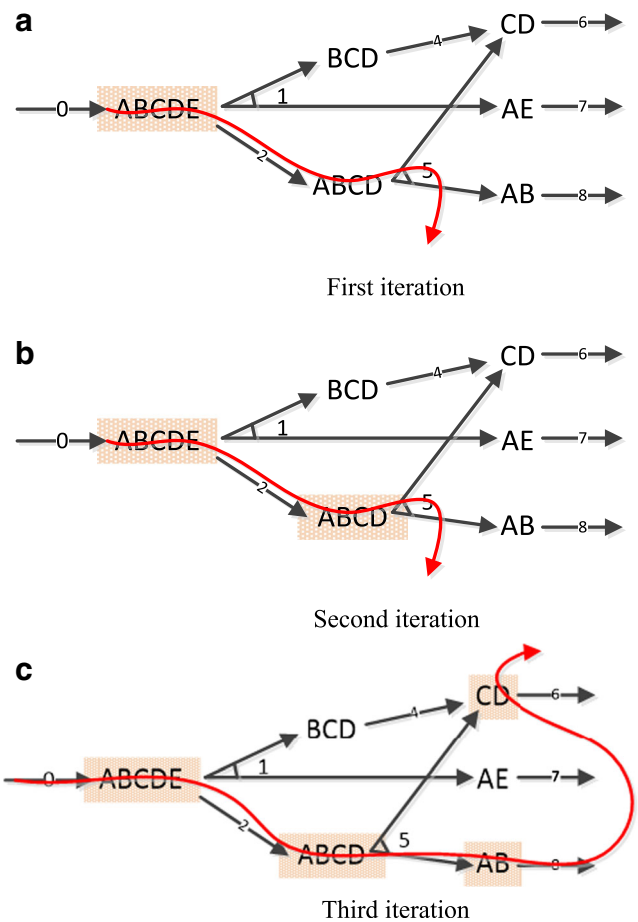

Fig. 17 *GoodDisassembility* example. **a** First iteration. **b** Second iteration. **c** Third iteration

nominal value of the "to be broken" *Part/Component* (the result of an irreversible disassembly operation) will be set to 0. After that, the same analysis as the *GoodDisassembility* will continue.

From the case study, it is evident that the disassembly planning is adaptive, which means, at each stage, the optimal disassembly sequence is optimal with regard to the current, updated information (for the part/component values). After one disassembly operation, this information is updated using the fuzzy ontology engine and is sent back to the LP model for next-round calculation. Also, the decision-making process is using the semantic reasoning capacities embedded in the DIM for detailed post disassembly strategy (reuse, recycle, or disposal).

5 Conclusion

This paper proposes an ontology-based disassembly information system which comprises an information model and a computational model for disassembly process planning. The computational model proposed in this paper focuses on tackling the uncertainty issue in a disassembly planning. It is a novel approach as it utilizes the power of semantic reasoning embedded in the proposed disassembly information model, and it combines the traditional disassembly LP model with fuzzy logic to generate disassembly sequences adaptively.

This work is a preliminary attempt for a comprehensive ontology-based product information model development and utilization. Disassembly, as one of the core stages in a product life cycle, has been studied here as a prototype, and the framework proposed in this paper can be applied into a more general domain like overall product design, so that the computer-aided product design (CAPD) can reach a new level.

References

1. Fenves SJ (2001) A core product model for representing design information, National Institute of Standards and Technology, NISTIR 6736. Gaithersburg, MD20899, USA
2. Fenves SJ, Foufou S, Bock C, Bouillon N, Sriram RD (2004) CPM2: a revised core product model for representing design information. National Institute of Standards and Technology, NISTIR 7185. Gaithersburg, MD20899, USA
3. Baysal MM, Roy U, Sudarsan R,Sriram RD, Lyons KW (2004) The open assembly model for exchange of assembly and tolerance information: overview and example. Proceedings of the DETC2004 ASME design engineering technical conferences. Salt Lake City, UT
4. Joung CB, Lee H, Ghodous P, Sriram RD, Feng SC, Kramer T (2011) Information model for disassembly for reuse, recycle, and remanufacturing. National Institute of Standards and Technology, NISTIR 7772. Gaithersburg, MD20899, USA
5. Zadeh LA (1975) The concept of a linguistic variable and its application to approximate reasoning I, II, II. Inf Sci 8:199–249–301–357, 9(1975), 43–80
6. Homem de Mello LS, Sanderson AC (1990) AND/OR graph representation of assembly plans. IEEE Trans Robot Autom 6(2):188–99
7. Tang Y, Zhou M (2000) Disassembly modeling, planning, and application: a review, proceedings of the 2000 IEEE, April
8. Fujiwara F, Suzuki T (1998) Integration of planning and scheduling for mechanical assembly based on timed petri net. Proceedings of Japan-US Symposium on Flexible Automation, Osaka
9. Moore KE, Gungor A, Gupta SM (1998) Petri net approach to disassembly process planning, computers. Ind Eng 35(1–2):165–8
10. Lambert AJD, Gupta SM (2005) Disassembly modeling for assembly, maintenance, reuse and recycling. CRC Press Boca Raton, Florida
11. Lambert AJD (1999) Linear programming in disassembly/clustering sequence generation. Comput Ind Eng 36(4):723–738
12. Lambert AJD (1997) Optimal disassembly of complex products. Int J Prod Res 35(9):2509–2523
13. Zussman E, Zhou MC (1999) A methodology for modeling and adaptive planning of disassembly processes. IEEE Trans Robot Autom 15:190–194
14. Martinez M, Pham VH, Favrel J (1997) Dynamic generation of disassembly sequences. In proceedings of 6th International Conference on Engineering technologies and factory automation, pp17-182
15. Geiger D, Zussman E, Lenz E (1996) Probabilistic reactive disassembly planning. CIRP Ann Manuf Technol 45:49–52
16. Guangdong T, MengChu Z (2012) Probability evaluation models of product cost subject to random removal time and different removal labor cost. IEEE Trans Autom Sci Eng 9:(2) April
17. Bobillo F, Straccia U (2011) Fuzzy ontology representation using OWL 2. In Int J Approx Reason
18. Zhu B, Roy U (2013) Disassembly information model incorporating dynamic capabilities for disassembly sequence generation. Robot Comput Integr Manuf. doi:10.1016/j.rcim.2013.03.003i
19. Gruber TR (1993) A translation approach to portable ontology specification. Knowl Acquis 5(2):199–220
20. Zhu B, Roy U (2013) Uncertain information representation and its usage in disassembly modeling. ASME 2013 International Design Engineering Technical Conferences (IDETC) and Comput Inform Eng Conf, Portland, US